

SRE RAG Agent v3.3 — Guia de Execução Local

Passo a Passo (Step by Step) para Inicialização, Operação e Validação do Laboratório

Este documento reúne as 3 formas homologadas de subir e operar o ambiente do **SRE Incident Simulation Lab v3.3** em ambiente local (WSL2 / Ubuntu ou Linux). Escolha a opção mais adequada ao seu objetivo de estudo.

Opção 1: Modo Desenvolvimento Local Rápido [MAIS RÁPIDO]

Ideal para desenvolvimento, testes de UI, simulação de incidentes e uso do War Room sem necessidade de inicializar contêineres Docker.

Passo 1: Navegue até o diretório raiz do projeto:

```
cd /home/magno/sre_rag_agent
```

Passo 2: Crie e ative o ambiente virtual Python 3.12 (caso ainda não exista):

```
python3 -m venv app/.venv
source app/.venv/bin/activate
```

Passo 3: Instale as dependências (com PyTorch CPU otimizado, sem CUDA bloat):

```
pip install --upgrade pip
pip install torch --index-url https://download.pytorch.org/whl/cpu
pip install -r app/requirements.txt
```

Passo 4: Inicie o servidor ASGI (Uvicorn) na porta 8080:

```
cd app
../app/.venv/bin/uvicorn main:app --host 0.0.0.0 --port 8080 --reload
# Ou execute diretamente na raiz: make dev
```

Passo 5: Acesse a interface e endpoints no navegador:

Dashboard Principal & Quiz	http://localhost:8080/dashboard
War Room (Terminal Interativo)	http://localhost:8080/dashboard#warroom
Conceitos SRE & Arquitetura	http://localhost:8080/dashboard#concepts
Métricas & Histórico	http://localhost:8080/dashboard#history
Health Check da Aplicação	http://localhost:8080/healthz

Opção 2: Modo Docker Compose (Multi-Network Isolado) [STACK SEGURA]

Sobe o ecossistema completo em contêineres: Nginx (Proxy reverso público na porta 80), FastAPI (API), PostgreSQL e Redis interligados pela rede interna isolada backend_net.

Passo 1: Execute o build e inicialize os serviços em background:

```
cd /home/magno/sre_rag_agent
docker compose up -d --build
# Ou use o atalho: make compose-up
```

Passo 2: Verifique a saúde e o status de inicialização dos contêineres:

```
docker compose ps
```

Passo 3: Acesso aos serviços via Nginx Reverse Proxy:

```
http://localhost/dashboard      # Interface Web (porta 80)
http://localhost/metrics        # Métricas Prometheus
```

Passo 4: Encerrar a stack quando finalizar os testes:

```
docker compose down
# Ou use o atalho: make compose-down
```

Opção 3: Modo Cluster Kubernetes (Kind + Helm) [PRODUÇÃO REAL]

Simulação completa de cluster K8s local com orquestração Helm, Prometheus Operator e dashboards Grafana pré-configurados.

Passo 1: Provisionar o cluster Kind e a stack de monitoramento:

```
cd /home/magno/sre_rag_agent
make setup
```

Passo 2: Realizar o deploy do SRE RAG Agent via Helm:

```
make deploy
```

Passo 3: Iniciar o encaminhamento de portas (Port-Forward):

```
make port-forward
```

Passo 4: Painéis e Credenciais de Acesso:

Aplicação SRE RAG	http://localhost:8080/dashboard	Sem auth
Prometheus	http://localhost:9090	Métricas PromQL
Grafana	http://localhost:3000	admin / admin
Alertmanager	http://localhost:9093	Alertas ativos

Passo 5: Desligar e limpar todos os recursos do cluster:

```
make clean
```

Validação Rápida da Suíte de Testes [QUALIDADE]

Para validar que todos os 16 cenários, o motor RAG e os runbooks estão íntegros:

```
make test  
# Executa: PYTHONPATH=.:app ./app/.venv/bin/pytest tests/ -v
```